

- Devoir surveillé de mi semestre - Jeudi 13 novembre 2025

<u>Groupe n° :</u>	<u>Nom et prénom chinois :</u>	<u>Prénom français :</u>	<u>Note /100 :</u>
--------------------	--------------------------------	--------------------------	--------------------

La notation tiendra compte du soin de la copie et de la rédaction.

Exercice n° 1 : /40 points

Partie I : Algorithmique

1. On cherche à écrire une fonction en Python, appelée `parcoursBFS`, de paramètres `M` de type `array`, et `s` de type `int`, qui parcourt en largeur le graphe G de matrice d'adjacences M à partir du sommet source s , les sommets de G étant numérotés $0, 1, \dots, n-1$, où n est la taille de la matrice M .

Compléter sur la feuille les « `***` » de la fonction Python ci-dessous afin qu'elle renvoie la liste des sommets accessibles depuis le sommet source s à l'aide d'un parcours en largeur du graphe G .

```
def parcoursBFS(M,s):
    n=len(M)
    Access= ***          # *** remplacé par :
    Avisiter= ***        # *** remplacé par :
    while *** :          # *** remplacé par :
        u=Avisiter.pop()
        L=[]
        for i in range(n):
            if *** :      # *** remplacé par :
                L.append(i)
        # L est la liste de tous les voisins
        # (ou successeurs) de u
        for k in L:
            if *** :      # *** remplacé par :
                ***        # *** remplacé par :
                Access.append(k)
    return(Access)
```

2. Dans cette question, on ne demande pas de justifications.

- (a) Quel est le nombre minimal de boucles `while` effectuées lorsque l'on exécute la fonction `parcoursBFS` ?

Pour $n \in \mathbb{N}^*$, donner un exemple de graphe orienté d'ordre n et d'un sommet source s associé pour lequel le nombre de boucles `while` effectuées est minimal.

Réponse :

Nombre minimal :

Représentation graphique d'un graphe orienté d'ordre n , avec pour sommet $s =$:

- (b) Quel est le nombre maximal de boucles `while` effectuées lorsque l'on exécute la fonction `parcoursBFS` ?

Pour $n \in \mathbb{N}^*$, donner un exemple de graphe orienté d'ordre n et d'un sommet source s associé pour lequel le nombre de boucles `while` effectuées est maximal.

Réponse :

Nombre maximal :

Représentation graphique d'un graphe orienté d'ordre n , avec pour sommet $s =$:

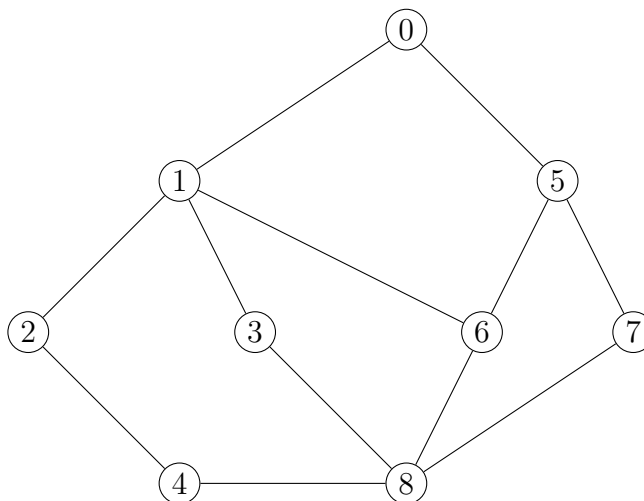
3. Les graphes en Python peuvent être définis sous la forme d'un dictionnaire, variable de type `dict`.

Compléter alors les « `***` » de la fonction Python ci-dessous, de paramètre `D` de type `dict`, afin qu'elle revoie la matrice d'adjacences associée au graphe défini par le dictionnaire `D`.

```
def traduction(D):
    n=len(D)
    L=D.keys()
    L=list(L)
    M= *** # *** remplacé par :
    for i in range(n):
        for j in range(n):
            u=L[i]
            v=L[j]
            if ****: # *** remplacé par :
                M[i,j]=1
    return(M)
```

Partie II : Un exemple

On considère le graphe G donné ci-contre :



4. Rappeler la définition du diamètre d'un graphe, puis préciser (sans justification) le diamètre de G .

Réponse :

5. Compléter le tableau ci-dessous qui donne l'évolution des listes **Access** et **Avisiter**, définies dans la fonction Python `parcoursBFS` de la partie I, lorsque l'on applique l'algorithme de parcours en largeur au graphe G en partant du sommet source $s = 0$.

Certains lignes pourront rester vides.

Étape initiale	Access=	Avisiter=
Fin de la boucle <code>while</code> n° 1	Access=	Avisiter=
Fin de la boucle <code>while</code> n° 2	Access=	Avisiter=
Fin de la boucle <code>while</code> n° 3	Access=	Avisiter=
Fin de la boucle <code>while</code> n° 4	Access=	Avisiter=
Fin de la boucle <code>while</code> n° 5	Access=	Avisiter=
Fin de la boucle <code>while</code> n° 6	Access=	Avisiter=
Fin de la boucle <code>while</code> n° 7	Access=	Avisiter=
Fin de la boucle <code>while</code> n° 8	Access=	Avisiter=
Fin de la boucle <code>while</code> n° 9	Access=	Avisiter=
Fin de la boucle <code>while</code> n° 10	Access=	Avisiter=

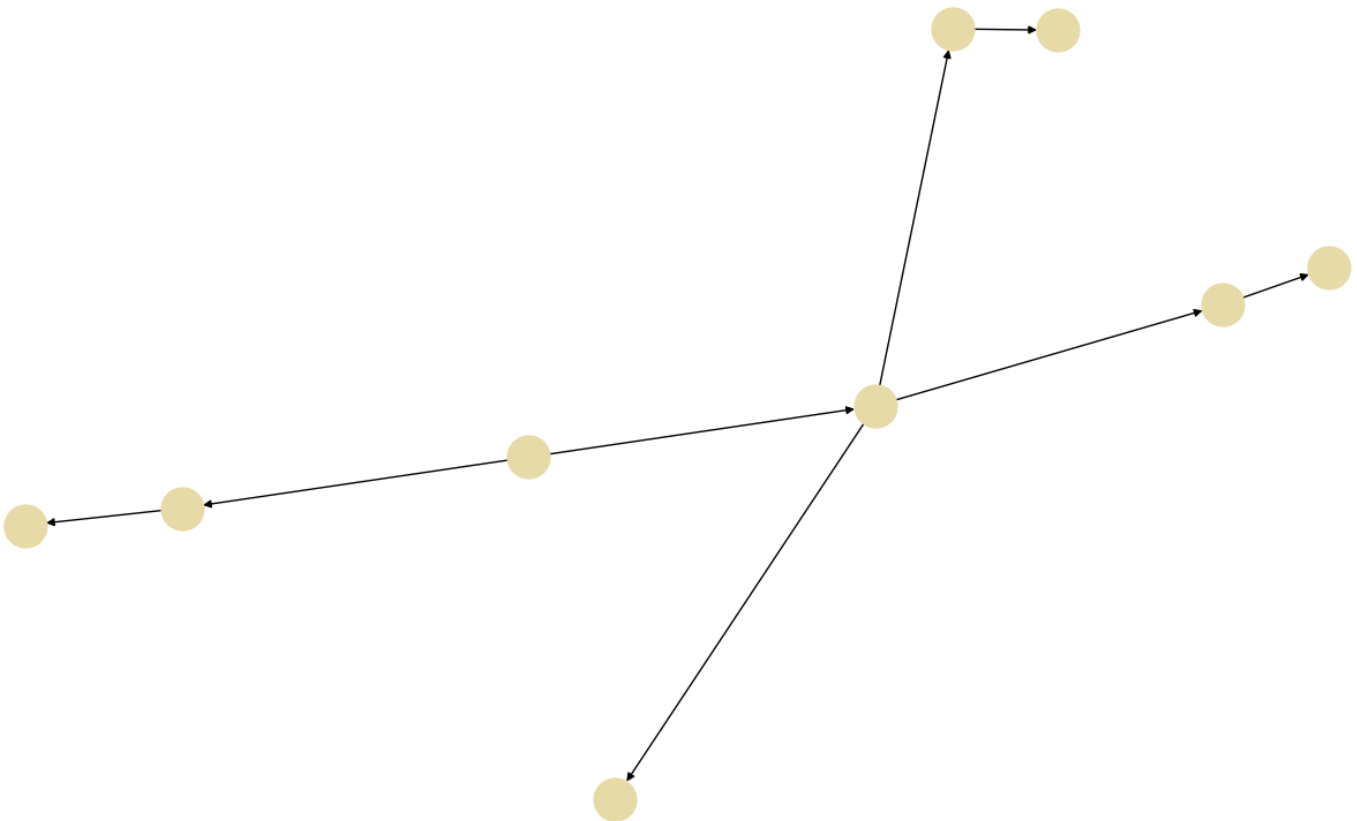
6. (a) Compléter les « `***` » du script en Python ci-après, utilisant la fonction `traduction` de la partie I, afin de définir le graphe G et d'appliquer l'algorithme BFS fourni par la bibliothèque `networkx` de Python.

```

import numpy as np
import networkx as nx
D= ***      # *** remplacé par :
M=traduction(D)
G=nx.Graph(M)
Gb=nx.bfs_tree(G,source=0)
nx.draw(Gb)

```

- (b) Après exécution, on obtient la fenêtre graphique ci-dessous.
Compléter le graphique en écrivant les numéros dans les sommets correspondants.

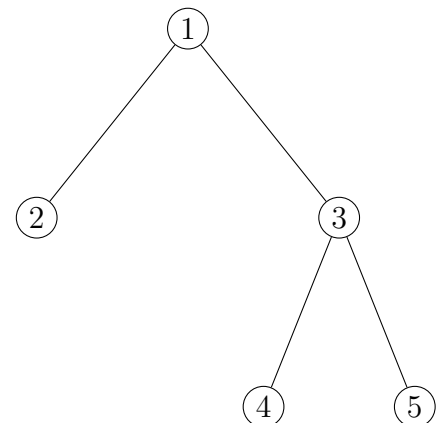


Exercice n° 2 : / 60 points

Partie I : Étude d'un exemple

Dans cette partie, on s'intéresse au graphe G donné ci-contre :

1. Justifier que G est un arbre.
2. On note Δ le maximum des degrés des sommets de G .
Préciser la valeur de Δ .
3. Écrire la matrice M d'adjacences associée au graphe G .



4. On note χ_M le polynôme caractéristique de la matrice M .

(a) Justifier qu'il existe deux réels a et b tels que

$$\forall \lambda \in \mathbb{R}, \quad \chi_M(\lambda) = \lambda^5 - 4\lambda^3 + a\lambda + b$$

(b) Montrer que $b = 0$.

(c) Montrer enfin que $a = 2$.

5. Déterminer les valeurs propres de la matrice M .

6. Vérifier que $\text{Sp}(M) \subset [-2\sqrt{\Delta-1}; 2\sqrt{\Delta-1}]$.

Partie II : Localisation des valeurs propres d'un arbre

Soit $G = (S, A)$ un arbre d'ordre n , avec $n \geq 3$ et $S = \{s_1, \dots, s_n\}$.

On note Δ le maximum des degrés des sommets de G , et on suppose que $\Delta \geq 2$.

7. Montrer que si u et v sont deux sommets distincts de G , alors il existe une unique chaîne élémentaire reliant u à v .

8. Montrer que si u et v sont deux sommets de G tels que $d(u, v) = \text{Diam}(G)$, alors les sommets u et v sont de degré 1.

Dans toute la suite de l'exercice, on note r un sommet de G tel que $\deg(r) = 1$.

Et on note :

- $\ell_r = 0$,
- pour tout sommet u de G distinct de r , ℓ_u la longueur de l'unique chaîne élémentaire reliant r à u .

9. Justifier que pour tout $u \in S$, $\ell_u = d(r, u)$.

10. Soit $s \in S - \{r\}$.

Montrer que si $\deg(s) \geq 2$, alors il existe un unique voisin t de s tel que $\ell_t = \ell_s - 1$, et $\ell_u = \ell_s + 1$ pour tout autre voisin u de s .

11. Soit $M = (m_{i,j})_{1 \leq i,j \leq n}$ une matrice carrée d'ordre n de $\mathcal{M}_n(\mathbb{R})$.

On suppose que λ une valeur propre réelle de M et on note $X = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}$ un vecteur propre associé.

Soit $k \in \llbracket 1, n \rrbracket$ tel que $|x_k| = \|X\|_\infty$, où $\|X\|_\infty = \max_{1 \leq i \leq n} |x_i|$.

En considérant la $k^{\text{ième}}$ ligne de l'égalité $MX = \lambda X$, montrer que : $|\lambda| \leq \max_{1 \leq i \leq n} \left(\sum_{j=1}^n |m_{i,j}| \right)$.

Dans la suite de cet exercice, on note :

- M la matrice d'adjacences de l'arbre G
- D la matrice diagonale, dont le coefficient diagonal situé à la $i^{\text{ième}}$ ligne est égal à $d_i = (\sqrt{\Delta-1})^{\ell_{s_i}}$
- D^{-1} la matrice inverse de D et $B = DMD^{-1}$.

12. Justifier que D^{-1} est bien définie et que $\text{Sp}(B) = \text{Sp}(M) \subset \mathbb{R}$.

13. Vérifier que : $\frac{\Delta}{\sqrt{\Delta-1}} \leq 2\sqrt{\Delta-1}$.

14. Expliciter les coefficients de la matrice $B = (b_{i,j})_{1 \leq i,j \leq n}$ en fonction de ceux de M .

15. À l'aide de la question 10, montrer que : $\forall i \in \llbracket 1, n \rrbracket, \sum_{j=1}^n |b_{i,j}| \leq 2\sqrt{\Delta-1}$.

16. Conclure que : $\text{Sp}(M) \subset [-2\sqrt{\Delta-1}; 2\sqrt{\Delta-1}]$.

Groupe n° :

Nom et prénom chinois :

Prénom français :

Résolution de l'exercice 2 :

Résolution de l'exercice 2 - suite 1 :

Résolution de l'exercice 2 - suite 2 :

Groupe n° : Nom et prénom chinois : Prénom français :

Résolution de l'exercice 2 - suite 4 :

Résolution de l'exercice 2 - suite 5 :

Résolution de l'exercice 2 - suite 6 :

Résolution de l'exercice 2 - suite 7 :